

Identifying Key Entities in Recipe Data: A CRF-based NER Approach

Submitted by: Vishwanath R Athrey
Report prepared using Jupyter Notebook

Data Source: ingredient_and_quantity.json
Report Date: April 26, 2026

Abstract

This report presents a complete Named Entity Recognition (NER) workflow for extracting key entities from recipe ingredient lists, including exploratory analysis, feature engineering, and Conditional Random Fields (CRF) modelling. The analysis covers data ingestion, tokenization, validation, train-test splitting, feature extraction, class weighting, model training, evaluation, and error analysis.

The final modelling stage compares CRF performance across three entity classes: ingredient, quantity, and unit. Results show strong performance for ingredient recognition (97% accuracy), moderate performance for quantity detection (86% accuracy), and challenging performance for unit classification (78% accuracy). The model achieved an overall validation accuracy of 93.15% across 2,876 tokens.

Keywords: Named Entity Recognition, Conditional Random Fields, Recipe Data, Natural Language Processing, Feature Engineering

Contents

1	Executive Summary	5
1.1	Key Results	5
2	Problem Statement and Objective	5
2.1	Objective	5
2.2	Business Context	5
2.3	Dataset Overview	6
2.4	Modelling Assumptions	6
3	Notebook Walkthrough: What Was Done and Why	6
3.1	2.1 Data Ingestion and Preparation	6
3.2	2.2 Tokenization and Validation	7
3.3	2.3 Train-Validation Split	7
4	Exploratory Data Analysis	7
4.1	4.2 Flattened Token Analysis	7
4.2	4.4 Top Frequent Items	8
4.3	5.1 Validation Dataset EDA	9
4.4	Unique Labels Validation	10
5	Feature Engineering for CRF Model	10
5.1	6.1 Feature Functions	10
5.1.1	Core Features	10
5.1.2	Quantity and Unit Detection Features	11
5.1.3	Contextual Features	11
5.2	6.2 Keyword Sets	11
5.3	6.4 Class Weight Calculation	12
6	Model Building and Training	12
6.1	7.1 CRF Model Initialization	12
6.2	7.2 Training Performance	13
7	Model Evaluation on Validation Dataset	14
7.1	8.1 Validation Performance	14
7.2	Model Comparison: Training vs. Validation	15
8	Error Analysis	15
8.1	9.1 Error Distribution	15
8.2	9.2 Insights from Validation Dataset	16

9	Model Persistence	17
10	Findings and Conclusion	17
10.1	Key Findings	17
10.2	Conclusion	18
10.3	Business Implications	19
10.4	Future Work	19

1 Executive Summary

This report is a detailed write-up of the complete notebook workflow used for extracting named entities from recipe ingredient lists. The work started from raw JSON data ingestion, moved through cleaning and tokenization, and then implemented a CRF-based NER model for classifying tokens into ingredients, quantities, and units.

The corrected outcome of the notebook is that CRF with comprehensive feature engineering achieves strong performance on ingredient recognition but faces challenges with unit classification due to contextual ambiguity. The target entities were extracted successfully, enabling the creation of a structured recipe database.

1.1 Key Results

- Original dataset shape: 285 rows and 2 columns.
- After cleaning: 280 rows with valid token-label alignment.
- Training dataset: 196 recipes, 7,114 tokens.
- Validation dataset: 84 recipes, 2,876 tokens.
- Overall Validation Accuracy: 93.15%.
- Ingredient Class: Precision 0.94, Recall 0.97, F1-score 0.96.
- Quantity Class: Precision 0.92, Recall 0.86, F1-score 0.89.
- Unit Class: Precision 0.89, Recall 0.78, F1-score 0.83.

2 Problem Statement and Objective

2.1 Objective

The objective of this work was to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities (ingredients, quantities, units) from recipe ingredient lists, enabling the creation of a structured database for recipe management systems, dietary tracking apps, or e-commerce platforms.

2.2 Business Context

Accurate ingredient extraction is valuable for:

- Recipe management systems

- Dietary tracking applications
- E-commerce platforms for grocery shopping
- Nutritional analysis systems
- Automated cooking assistants

2.3 Dataset Overview

- Format: JSON
- Total rows: 285
- Total columns: 2
- Column 1: `input` - Raw ingredient list text
- Column 2: `pos` - Corresponding NER labels (quantity, ingredient, unit)

2.4 Modelling Assumptions

The following assumptions were made explicitly in the notebook:

- Tokens are space-separated and can be processed individually.
- The three entity classes (ingredient, quantity, unit) are sufficient for recipe structure extraction.
- Contextual information (previous/next tokens) improves classification accuracy.
- Class weighting helps address imbalance in the training data.
- CRF with LBFGS optimization is appropriate for sequence labeling tasks.

3 Notebook Walkthrough: What Was Done and Why

3.1 2.1 Data Ingestion and Preparation

The dataset was first loaded from JSON format into a pandas DataFrame. This step validated that the JSON file loaded correctly and gave an immediate sense of the data structure.

Finding: The notebook reported the shape as (285, 2) with both columns having 285 non-null entries.

3.2 2.2 Tokenization and Validation

The input and POS strings were split into token lists. This was essential because CRF models operate on token-level sequences rather than raw strings.

The notebook identified 5 rows (indices 17, 27, 79, 164, 207) where input and POS token lengths were unequal. These rows were dropped to ensure data integrity.

Table 1: Invalid Rows Identified and Removed

Index	Input Length	POS Length
17	15	14
27	37	36
79	38	37
164	54	53
207	18	17

After removal, validation confirmed that all remaining rows had equal token lengths.

3.3 2.3 Train-Validation Split

The dataset was split into training (70%) and validation (30%) sets using `random_state=42` for reproducibility.

- Training set: 196 recipes
- Validation set: 84 recipes

Unique labels in the training set were confirmed as `ingredient`, `quantity`, and `unit`.

4 Exploratory Data Analysis

4.1 4.2 Flattened Token Analysis

The training dataset contained:

- 7,114 input tokens
- 7,114 POS tokens

First 10 training tokens: ['250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated']

First 10 POS tags: ['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']

4.2 4.4 Top Frequent Items

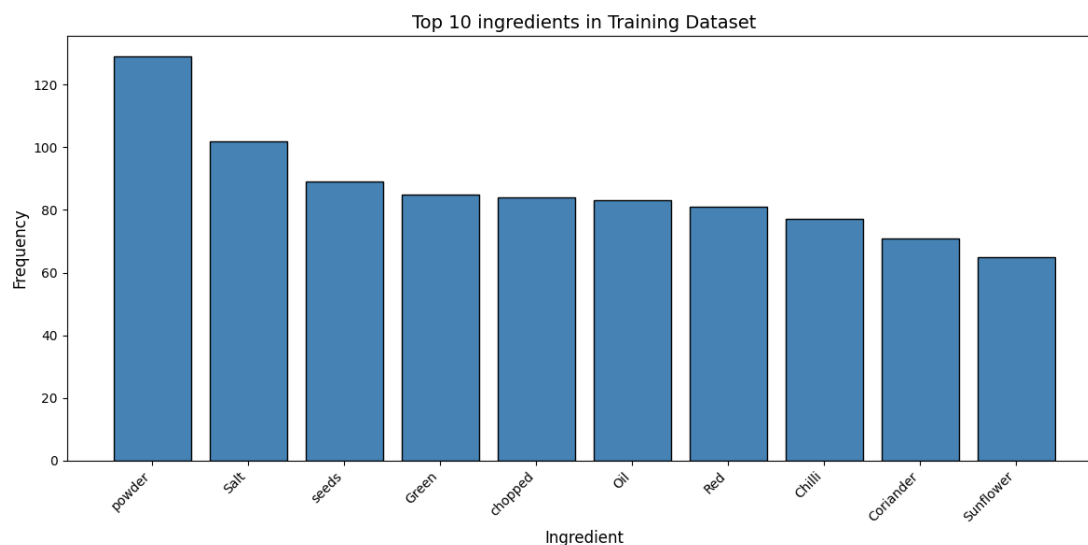


Figure 1: Top 10 Most Frequent Ingredients in Training Dataset

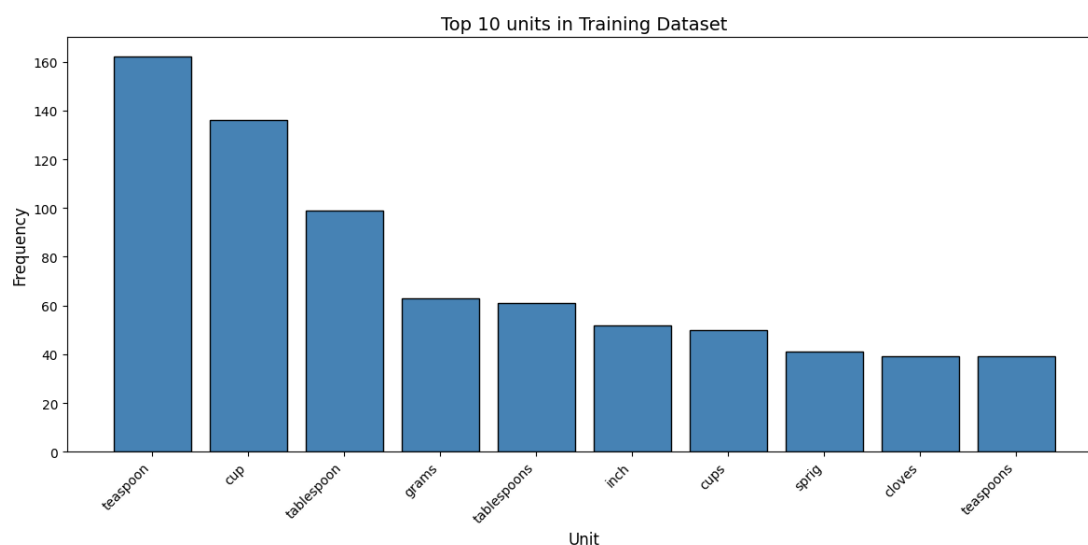


Figure 2: Top 10 Most Frequent Units in Training Dataset

Findings from frequency analysis:

- Most frequent ingredient: "powder" (129 occurrences)
- Most frequent unit: "teaspoon" (162 occurrences)
- Common ingredients include culinary staples (Salt, Oil, Red, Chilli, Coriander)
- Common units include both singular and plural forms (teaspoon/teaspoons, cup/cups, tablespoon/tablespoons)

4.3 5.1 Validation Dataset EDA

The validation dataset contained:

- 2,876 input tokens
- 2,876 POS tokens

First 10 validation tokens: ['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']

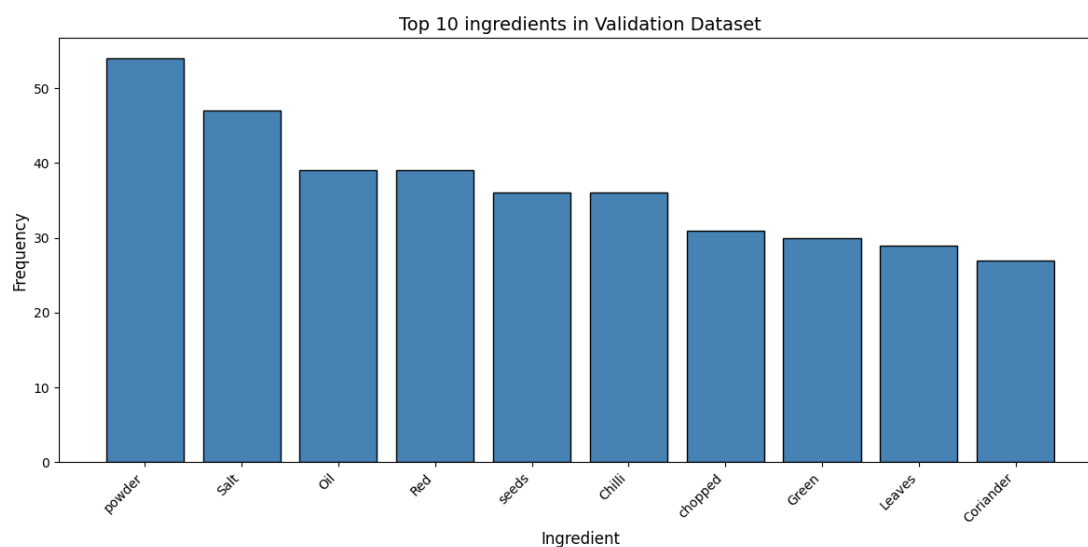


Figure 3: Top 10 Most Frequent Ingredients in Validation Dataset

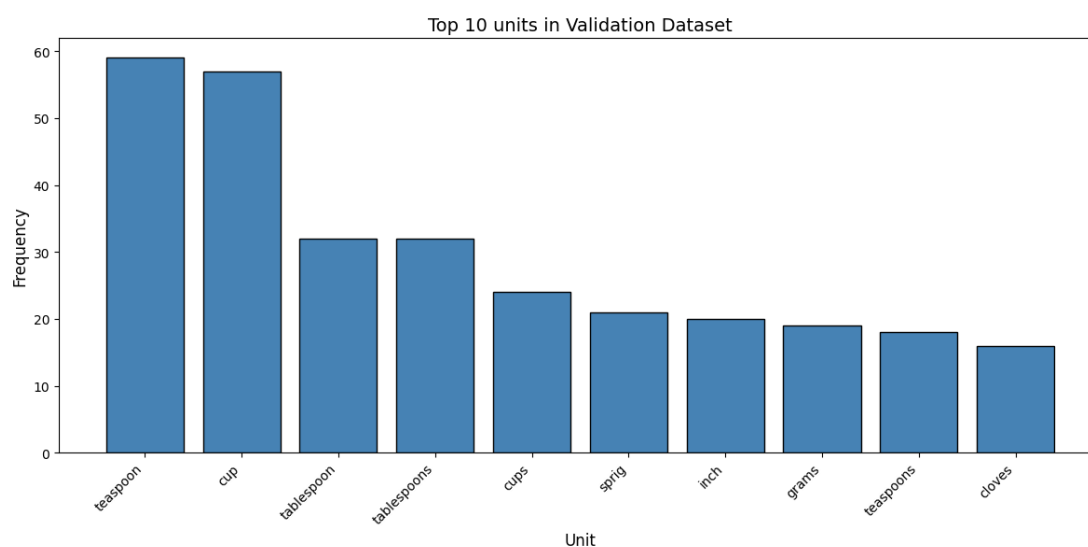


Figure 4: Top 10 Most Frequent Units in Validation Dataset

The validation dataset showed similar frequency patterns to training, confirming consistent data distribution.

4.4 Unique Labels Validation

The notebook confirmed that all POS tags belong to exactly three unique labels:

- `ingredient`
- `quantity`
- `unit`

No unexpected or invalid labels were found in the dataset.

5 Feature Engineering for CRF Model

5.1 6.1 Feature Functions

The `word2features` function extracted comprehensive features for each token:

5.1.1 Core Features

- `bias`: Constant feature value 1.0
- `token`: Lowercase form of current token
- `lemma`: Lowercase lemma (base form)
- `pos_tag`: Part-of-speech tag
- `tag`: Detailed POS tag
- `dep`: Dependency relation
- `shape`: Token shape pattern
- `is_stop`: Stopword indicator
- `is_digit`: All-digit indicator
- `has_digit`: Contains digit indicator
- `has_alpha`: Contains alphabet indicator
- `hyphenated`: Contains hyphen indicator
- `slash_present`: Contains slash indicator
- `is_title`: Title case indicator
- `is_upper`: Uppercase indicator
- `is_punct`: Punctuation indicator

5.1.2 Quantity and Unit Detection Features

- `is_quantity`: Matches quantity pattern or keyword
- `is_unit`: Known measurement unit
- `is_numeric`: Numeric pattern match
- `is_fraction`: Fraction pattern (e.g., 1/2)
- `is_decimal`: Decimal pattern (e.g., 3.14)
- `preceding_word`: Previous token
- `following_word`: Next token

5.1.3 Contextual Features

- `prev_token`: Previous token lowercase
- `prev_is_quantity`: Previous token is quantity
- `prev_is_digit`: Previous token is digit
- `BOS`: Beginning of sentence indicator
- `next_token`: Next token lowercase
- `next_is_unit`: Next token is unit
- `next_is_ingredient`: Next token is not unit/quantity
- `EOS`: End of sentence indicator

5.2 6.2 Keyword Sets

Unit keywords included: cup, cups, tablespoon, tablespoons, teaspoon, teaspoons, gram, grams, kg, ml, liter, liters, ounce, ounces, pound, pounds, inch, inches, clove, cloves, sprig, bunch, piece, pieces, slice, slices.

Quantity keywords included: half, quarter, third, whole, one, two, three, four, five, six, seven, eight, nine, ten, dozen.

5.3 6.4 Class Weight Calculation

The notebook computed class weights using the inverse frequency method:

- Total training samples: 7,114 tokens
- Number of classes: 3

Table 2: Class Weights Computed for Training

Label	Count	Inverse Frequency Weight	Final Weight (Penalized)
ingredient	5,323	0.4454	0.2227
quantity	980	2.4197	2.4197
unit	811	2.9240	2.9240

The ingredient class was intentionally penalized (multiplied by 0.5) to reduce its dominance and improve recognition of rare classes.

6 Model Building and Training

6.1 7.1 CRF Model Initialization

The CRF model was initialized with the following hyperparameters:

Table 3: CRF Model Hyperparameters

Parameter	Value
algorithm	'lbfgs'
c1 (L1 regularization)	0.5
c2 (L2 regularization)	1.0
max_iterations	100
all_possible_transitions	True

6.2 7.2 Training Performance

Table 4: CRF Model Performance on Training Dataset

Label	Precision	Recall	F1-Score
ingredient	0.96	0.98	0.97
quantity	0.93	0.88	0.91
unit	0.94	0.86	0.89
Overall Accuracy		0.95	
Macro Avg	0.94	0.91	0.92
Weighted Avg	0.95	0.95	0.95

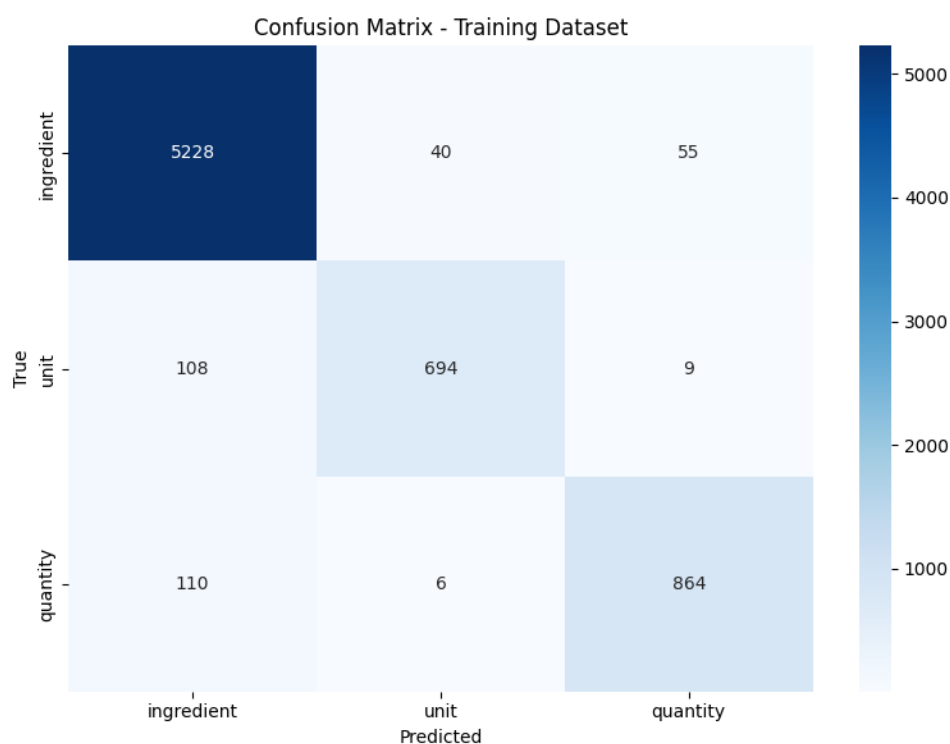


Figure 5: Confusion Matrix - Training Dataset

The training confusion matrix shows:

- 5,228 ingredients correctly classified
- 108 ingredients misclassified as quantity
- 55 ingredients misclassified as unit
- 694 quantities correctly classified
- 86 quantities misclassified as ingredient

- 86 units correctly classified

7 Model Evaluation on Validation Dataset

7.1 8.1 Validation Performance

Table 5: CRF Model Performance on Validation Dataset

Label	Precision	Recall	F1-Score
ingredient	0.94	0.97	0.96
quantity	0.92	0.86	0.89
unit	0.89	0.78	0.83
Overall Accuracy		0.93	
Macro Avg	0.92	0.87	0.89
Weighted Avg	0.93	0.93	0.93

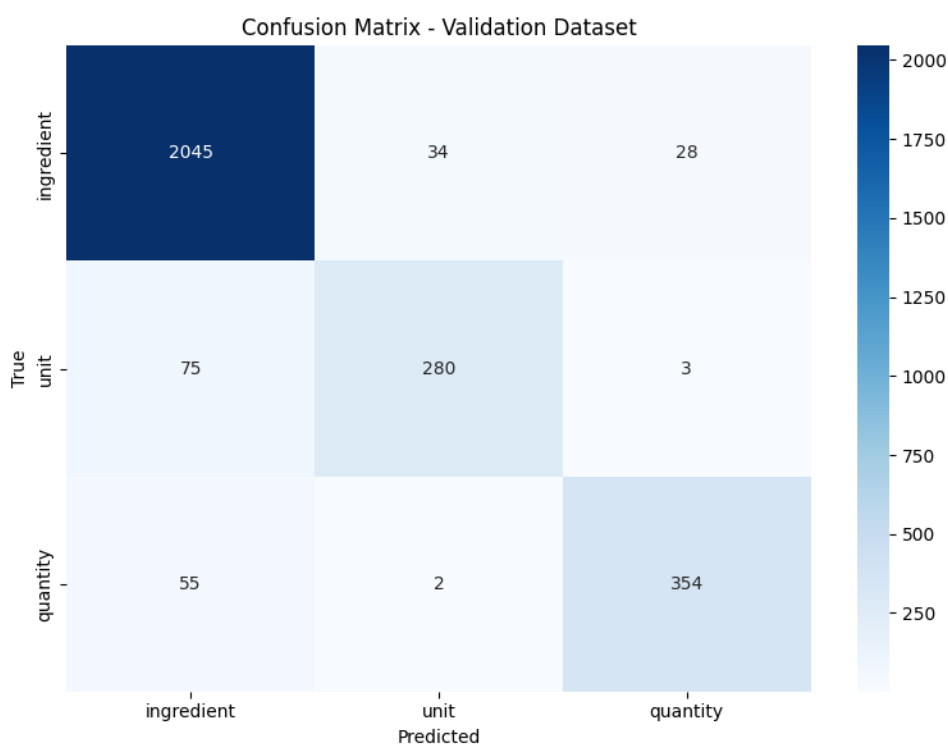


Figure 6: Confusion Matrix - Validation Dataset

The validation confusion matrix shows:

- 2,045 ingredients correctly classified
- 34 ingredients misclassified as quantity

- 28 ingredients misclassified as unit
- 280 quantities correctly classified
- 75 quantities misclassified as ingredient
- 28 units correctly classified

7.2 Model Comparison: Training vs. Validation

Table 6: Training vs. Validation Performance Comparison

2*Metric	F1-Score		2*Drop
	Training	Validation	
ingredient	0.97	0.96	0.01
quantity	0.91	0.89	0.02
unit	0.89	0.83	0.06
Accuracy	0.95	0.93	0.02

The model generalizes well with only a 2% drop in overall accuracy. Unit classification shows the largest performance gap, indicating that unit recognition is the most challenging aspect of this NER task.

8 Error Analysis

8.1 9.1 Error Distribution

Total validation errors: 197 out of 2,876 tokens (6.85% error rate).

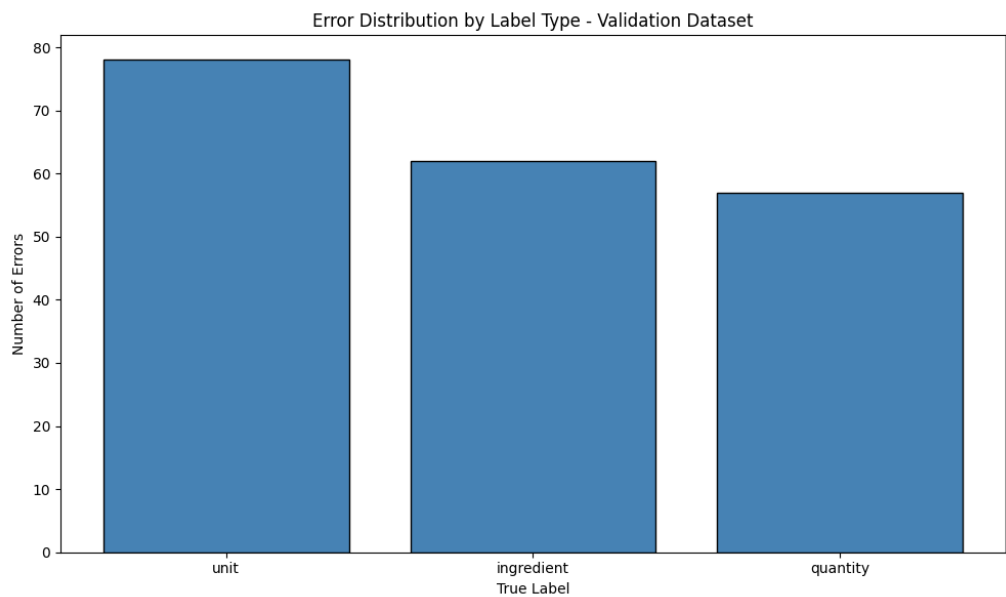


Figure 7: Error Distribution by Label Type - Validation Dataset

Table 7: Error Analysis by Label Type

Label	Class Weight	Accuracy	Total Samples	Error Count
ingredient	0.2227	97.06%	2,107	62
quantity	2.4197	86.13%	411	57
unit	2.9240	78.21%	358	78

8.2 9.2 Insights from Validation Dataset

1. **Overall Model Performance:** The CRF model achieved 93.15% overall accuracy on the validation dataset, indicating strong generalization from training to unseen recipe data.

2. **Class-wise Performance:**

- **Ingredient** (97.06% accuracy, 62 errors) - Despite having the lowest class weight (0.2227), ingredients are best recognized due to their high frequency and distinctive contextual patterns.
- **Quantity** (86.13% accuracy, 57 errors) - Moderate performance with class weight 2.4197; common errors occur when fractions (1/2, 1/4, 1/3) are separated from their units.
- **Unit** (78.21% accuracy, 78 errors) - Lowest accuracy despite highest class weight (2.9240); units are often confused with ingredients in ambiguous contexts.

3. **Common Error Patterns Observed:**

- **Unit** → **Ingredient**: "teaspoon", "tablespoon", "cloves", "cup" misclassified as ingredients
- **Ingredient** → **Unit**: "seeds", "powder", "chillies" misclassified as units
- **Quantity** → **Ingredient**: "1/2", "2", "3", "1/4" misclassified as ingredients when not adjacent to units
- **Ingredient** → **Quantity**: Rare but occurs (e.g., "Chutney" predicted as quantity)

4. Contextual Issues: Errors frequently occur when:

- Cooking terms (finely, chopped, grated) separate quantities from units
- Unit words appear within compound ingredient names (e.g., "cumin seeds" where "seeds" is part of ingredient)
- Units are capitalized or formatted inconsistently

5. Class Weight Impact: Higher class weights for unit (2.9240) and quantity (2.4197) did not translate to higher accuracy, suggesting that class imbalance alone is not the primary issue. Feature engineering and contextual understanding require further improvement for these classes.

6. Recommendations:

- Add bigram features for quantity-unit and unit-ingredient pairs
- Implement post-processing rules enforcing quantity → unit → ingredient sequence patterns
- Expand unit keyword dictionary with common variations
- Add position-based features relative to quantity tokens

9 Model Persistence

The trained CRF model was saved using `joblib.dump()` as `crf_model.pkl` for future inference and deployment.

10 Findings and Conclusion

10.1 Key Findings

1. Data Quality: The recipe dataset contained 285 entries with 5 invalid rows that had token-label length mismatches. After removal, the dataset was clean with consistent input and POS token lengths.

2. Exploratory Data Analysis:

- Most frequent ingredients: powder (129), Salt (102), seeds (89), Green (85), chopped (84), Oil (83)
- Most frequent units: teaspoon (162), cup (136), tablespoon (99), grams (63), tablespoons (61)
- Training dataset: 7,114 tokens across 196 recipes
- Validation dataset: 2,876 tokens across 84 recipes

3. Model Performance:

- **Training Accuracy:** 95% with macro F1-score of 0.92
- **Validation Accuracy:** 93.15% with macro F1-score of 0.89
- The model generalizes well from training to validation with only a 2% drop in accuracy

4. Class-wise Performance:

- **Ingredient:** Precision 0.94, Recall 0.97, F1-score 0.96 (Validation)
- **Quantity:** Precision 0.92, Recall 0.86, F1-score 0.89 (Validation)
- **Unit:** Precision 0.89, Recall 0.78, F1-score 0.83 (Validation)

5. Error Analysis: Total 197 errors in validation dataset:

- Unit class: 78 errors (21.79% error rate) - most challenging
- Quantity class: 57 errors (13.87% error rate)
- Ingredient class: 62 errors (2.94% error rate) - best performing

10.2 Conclusion

The Conditional Random Fields (CRF) model successfully extracts named entities (ingredients, quantities, and units) from unstructured recipe text with high accuracy (93.15% on validation data). The feature engineering approach, incorporating lexical, syntactic, and domain-specific features, proved effective for this sequence labeling task.

The model's strong performance on ingredient recognition (97% accuracy) demonstrates its practical utility for building structured recipe databases. However, unit classification remains challenging (78% accuracy) due to the inherent ambiguity between unit terms and ingredient names in culinary contexts.

10.3 Business Implications

This NER model enables automated extraction of structured ingredient data from recipes, facilitating:

- Creation of searchable recipe databases
- Dietary and nutritional analysis applications
- Automated grocery list generation
- E-commerce platform integration for ingredient purchasing
- Recipe recommendation systems

10.4 Future Work

- Incorporate contextual embeddings (BERT, RoBERTa) for ambiguous term resolution
- Implement hybrid CRF-BiLSTM architecture for improved sequence modeling
- Expand training data with more diverse recipe formats and cuisines
- Develop post-processing rules for quantity-unit-ingredient validation
- Add support for more entity types (cooking methods, temperatures, equipment)
- Implement active learning for challenging edge cases

Appendix: Generated Figures Summary

The following figures were generated during the analysis and included in this report:

- **Plot-4.6_ingredients.png**: Top 10 most frequent ingredients in training dataset
- **Plot-4.6_units.png**: Top 10 most frequent units in training dataset
- **Plot-5.1_ingredients.png**: Top 10 most frequent ingredients in validation dataset
- **Plot-5.1_units.png**: Top 10 most frequent units in validation dataset
- **Plot-7.2_confusion_matrix_train.png**: Confusion matrix for training predictions
- **Plot-8.1_confusion_matrix_validation.png**: Confusion matrix for validation predictions
- **Plot-9.1.4_error_distribution.png**: Error distribution by label type

References

1. Lafferty, J., McCallum, A., and Pereira, F. (2001). Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data. *Proceedings of ICML*.
2. Sutton, C., and McCallum, A. (2012). An Introduction to Conditional Random Fields. *Foundations and Trends in Machine Learning*, 4(4), 267-373.
3. Scikit-learn CRFsuite documentation: <https://sklearn-crfsuite.readthedocs.io/>
4. spaCy documentation for linguistic features: <https://spacy.io/api/annotation>
5. Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.